

NumPy Indexing & Shapes

Multiple Choice Questions

Instructions: Answer all questions. Each question has one correct answer.

Q1: What is the shape of the array?

```
x = np.array([[[[1],[2],[3]], [[4],[5],[6]]]])
```

`x.shape`

- A) (2, 3)
- B) (2, 3, 1)
- C) (6, 1)
- D) (3, 2, 1)

Q2: What will be the output of `x[1:2]`?

```
x = np.array([[[[1],[2],[3]], [[4],[5],[6]]]])
```

`x[1:2]`

- A) [4, 5, 6]
- B) `array([[[4], [5], [6]]])`
- C) `array([[[4], [5], [6]]])`
- D) `IndexError`

Q3: What is the result of fancy indexing with negative indices?

```
x = np.arange(10, 1, -1)
```

```
idx = np.array([0, -1, 4, -4, 2])
```

`x[idx]`

- A) `array([10, 2, 6, 5, 8])`
- B) `array([10, 9, 6, 5, 8])`
- C) `array([10, 2, 6, 6, 8])`
- D) `IndexError`

Q4: What is the shape of `x[[0, 2, 1]]`?

```
x = np.array([[1, 2], [3, 4], [5, 6]])
```

```
result = x[[0, 2, 1]]
```

- A) (3, 2)
- B) (2, 3)
- C) (3,)
- D) (6,)

Q5: What will `x[np.array([1, -1])]` return?

```
x = np.array([[1, 2], [3, 4], [5, 6]])
```

```
x[np.array([1, -1])]
```

- A) `array([[3, 4], [1, 2]])`
- B) `array([[3, 4], [5, 6]])`
- C) `IndexError`
- D) `array([[5, 6], [3, 4]])`

Q6: What error occurs and why?

```
x = np.array([[1, 2], [3, 4], [5, 6]])
```

```
x[np.array([3, 4])]
```

- A) `ValueError: array is incompatible`
- B) `IndexError: index 3 is out of bounds for axis 0 with size 3`
- C) `IndexError: index 4 is out of bounds for axis 0 with size 3`
- D) `TypeError: advanced indexing requires integer array`

Q7: What is the output?

```
x = np.array([10, 20, 30, 40, 50])
```

```
x[[0, 2, 4]]
```

- A) `array([10, 30, 50])`
- B) `[10, 30, 50]`
- C) `array([10, 20, 30, 40, 50])`
- D) `IndexError`

Q8: What will be the output?

```
x = np.array([5, 4, 3, 2, 1])  
idx = np.array([-1, -2, -3])  
x[idx]
```

- A) array([1, 2, 3])
- B) array([5, 4, 3])
- C) array([1, 2, 3])
- D) IndexError: index -1 is out of bounds

Q9: What is the shape of the result?

```
x = np.arange(12).reshape(3, 4)  
result = x[np.array([0, 2])]
```

- A) (2,)
- B) (2, 4)
- C) (3, 2)
- D) (6,)

Q10: Which indexing method is being used?

```
x = np.array([10, 20, 30, 40, 50])  
indices = np.array([1, 3])  
result = x[indices]
```

- A) Boolean indexing
- B) Slicing
- C) Fancy indexing
- D) Simple indexing

Q11: What will this return?

```
x = np.array([[1, 2, 3], [4, 5, 6]])  
x[np.array([1, 0])]
```

- A) array([[1, 2, 3], [4, 5, 6]])
- B) array([[4, 5, 6], [1, 2, 3]])

- C) `array([4, 1])`
- D) `IndexError`

Q12: What is `x[-1]`?

```
x = np.arange(1, 11)
x[-1]
```

- A) 1
- B) 10
- C) 9
- D) `IndexError`

Q13: What error occurs?

```
x = np.array([[1, 2], [3, 4]])
x[np.array([0, 1, 2])]
```

- A) `IndexError: index 2 is out of bounds for axis 0 with size 2`
- B) `ValueError: shape mismatch`
- C) `TypeError: integer required`
- D) No error, returns `array([[1, 2], [3, 4], [1, 2]])`

Q14: What is the output?

```
x = np.array([100, 200, 300, 400, 500])
x[np.array([4, 1, 3, 0])]
```

- A) `array([100, 200, 300, 400, 500])`
- B) `array([500, 200, 400, 100])`
- C) `array([400, 300, 200, 100])`
- D) `IndexError`

Q15: What is the result?

```
x = np.arange(6).reshape(2, 3)
x[[0]]
```

- A) `array([0, 1, 2])`

- B) `array([[0, 1, 2]])`
- C) 0
- D) `IndexError`

Q16: What is the shape of `x[0]`?

```
x = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  
result = x[0]
```

- A) (1, 2, 2)
- B) (2, 2)
- C) (4,)
- D) (2,)

Q17: What is the shape after this indexing?

```
x = np.arange(24).reshape(2, 3, 4)  
result = x[1, :, 2]
```

- A) (3,)
- B) (4,)
- C) (2, 3)
- D) (1, 3)

Q18: What is the shape of `x[:,2]`?

```
x = np.arange(20).reshape(4, 5)  
result = x[:,2]
```

- A) (2, 5)
- B) (4, 5)
- C) (2, 2)
- D) (5,)

Q19: What is the shape after fancy indexing?

```
x = np.arange(20).reshape(4, 5)  
result = x[np.array([0, 2, 3])]
```

- A) (3, 5)
- B) (3,)
- C) (5,)
- D) (15,)

Q20: What is the shape of `x[1:3, 2:4]`?

```
x = np.arange(30).reshape(5, 6)
result = x[1:3, 2:4]
```

- A) (2, 2)
- B) (3, 4)
- C) (5, 6)
- D) (4,)

Q21: What is the shape?

```
x = np.zeros((3, 4, 5))
result = x[:, 1, :]
```

- A) (3, 5)
- B) (4, 5)
- C) (3, 4)
- D) (1, 3, 5)

Q22: What is the shape of `x[np.array([0, 2]), np.array([1, 3])]`?

```
x = np.arange(20).reshape(4, 5)
result = x[np.array([0, 2]), np.array([1, 3])]
```

- A) (2, 5)
- B) (4, 2)
- C) (2,)
- D) (2, 2)

Q23: What is the shape after this operation?

```
x = np.arange(60).reshape(3, 4, 5)
```

```
result = x[1]
```

- A) (4, 5)
- B) (3, 4)
- C) (5,)
- D) (1, 4, 5)

Q24: What is the shape of `x[:2, :, 1]`?

```
x = np.arange(60).reshape(3, 4, 5)
```

```
result = x[:2, :, 1]
```

- A) (2, 4)
- B) (2, 5)
- C) (3, 4)
- D) (2, 4, 1)

Q25: What is the shape after fancy indexing on a 3D array?

```
x = np.arange(24).reshape(2, 3, 4)
```

```
result = x[np.array([0, 1])]
```

- A) (2, 3, 4)
- B) (2, 4)
- C) (3, 4)
- D) (2,)

Q26: What is the shape of `x[-1]`?

```
x = np.arange(12).reshape(3, 4)
```

```
result = x[-1]
```

- A) (1, 4)
- B) (4,)
- C) (3,)
- D) (3, 4)

Q27: What is the resulting shape?

```
x = np.arange(24).reshape(2, 3, 4)  
result = x[:, np.array([0, 2]), :]
```

- A) (2, 4)
- B) (2, 2, 4)
- C) (3, 4)
- D) (2, 3, 4)

Q28: What is the shape of `x[0:2, 1:4]`?

```
x = np.arange(30).reshape(5, 6)  
result = x[0:2, 1:4]
```

- A) (2, 3)
- B) (2, 6)
- C) (5, 3)
- D) (3, 3)

Q29: What is the shape after this operation?

```
x = np.ones((4, 5, 6))  
result = x[:, :2, :3, :]
```

- A) (2, 3, 6)
- B) (4, 3, 6)
- C) (2, 5, 6)
- D) (2, 3, 3)

Q30: What is the shape?

```
x = np.arange(120).reshape(2, 3, 4, 5)  
result = x[1, :, 0, :]
```

- A) (3, 5)
- B) (4, 5)
- C) (2, 3)
- D) (1, 3, 5)